

CAS4133 LLM · 기말 대비 · Chapter #11

11장. LLM 에이전트 (LLM Agents)

LLM을 brain으로 삼아 Perception·Planning & Action·Tools·Memory를 갖춘 에이전트 — ReAct/Reflexion/ToT, MemGPT/A-Mem, Multi-agent까지 3개 강의에 걸친 대형 챕터

 강의일: 05/20 · 05/27 · 06/01 (3개 강의) 예상 비중: 매우 높음 (기말 핵심) 분량: 본문 9개 섹션 + 예상문제 17

목차

1. LLM Agent의 정의와 핵심 구성요소 (Perception · Reasoning · Action)
2. Perception 확장 — ViT 패치 임베딩, LLaVA, VideoPoet, VLA
3. Planning & Action ① — ReAct (reasoning + acting interleave)
4. Planning & Action ② — Reflexion (verbal RL)
5. Planning & Action ③ — Plan-and-Solve & Tree of Thoughts (+4종 비교표)
6. Multi-modal Action — Diffusion 결합 (MetaQueries, BLIP3-o)
7. Memory ① — MemGPT (OS-inspired memory)
8. Memory ② — A-Mem (Agentic Memory) + MemGPT vs A-Mem 비교
9. Multi-agent System — MDAgents · Orchestration · Routing
10. 예상문제 (T/F 10 + Pick-all 7)

1

LLM Agent의 정의와 핵심 구성요소 (Key Concepts)

Agentic AI(multi-agent system)는 시장 규모·연구 분야 양면에서 현재 AI의 가장 뜨거운 주제다. MMLU·GPQA 같은 단순 추론 벤치마크는 최신 LLM에게 거의 풀린 문제(solved)로 간주되고, 대기업들의 평가는 **agentic 벤치마크**(예: SWE-bench)로 옮겨가고 있다 (DeepSeek-V3.2, Claude Opus 4.5 발표 등에 반영).

정의

Multi-agent system: "multiple intelligent agents"로 구성되어, **개별 에이전트 하나로는 불가능한(impossible for individual agent)** 문제를 풀 수 있는 시스템.

Intelligent agent: 환경을 **지각(perceive)**하고, **행동(action)**을 취하며, 이를 반복해 **스스로 성능을 개선(improving)**할 수 있는 개체. 지각과 행동 사이에 암묵적으로 **추론(reasoning)**이 존재
→ 핵심 3요소는 **Perception, Reasoning, Action**.

교수 강조 (강의 발언)

"최근 'agent system'이라고 말할 때의 정의는 사실상 **LLM이 에이전트 시스템을 구동(drive)한다**는 것을 전제합니다. 즉 LLM을 지능형 에이전트의 brain으로 간주합니다."

LLM Agent = LLM + (1) Planning & Action + (2) Tools + (3) Memory

LLM의 기본 역량(언어 이해, instruction following, reasoning)에 더해, 에이전트가 되려면 세 가지 추가 역량이 필요하다 (Lilian Weng 블로그 기준):

- 1 **Planning & Action** — 문제를 계획하고 환경에 행동을 취하는 능력 (ReAct, Reflexion, Plan-and-Solve, ToT)
- 2 **Tools** — 외부 도구 호출 (#10 Tool Use 챕터에서 다룸)
- 3 **Memory** — 컨텍스트 윈도우를 넘는 경험의 외부 저장·선택적 검색 (MemGPT, A-Mem)

★ 시험 핵심

슬라이드에 반복 강조된 문장: **"Not simply improved by just scaling-up model & data size"** — 이런 추가 역량은 모델·데이터를 키운다고 저절로 개선되지 않으며, 새 학습 방법·새 데이터

유형·LLM 주위의 새로운 harness 설계가 필요하다. 메모리는 LLM 내부에 통합하기 가장 어려운 부분(외부 시스템으로 처리), tool use는 학습으로 통합 가능.

⚠ 함정 포인트 (T/F 대비)

- "Agentic capabilities (planning, tool use, memory) naturally emerge by simply scaling up model and data size." → **False**. The slide explicitly states they are *not* simply improved by scaling-up; new training methods/data/harnesses are needed.
- "A multi-agent system is any system that uses an LLM more than once." → **False**. The definition requires multiple intelligent agents solving problems that are impossible for an individual agent.
- "An intelligent agent only needs perception and action." → **False**. Reasoning (implicit between perception and action) and the ability to improve its performance through repetition are also part of the definition.

예시로 보는 Perception·Reasoning·Action: VLA 모델

Vision-Language-Action (VLA) 모델 (예: **OpenVLA**, Kim et al., arXiv 24.06)은 에이전트의 가장 단순·자연스러운 형태다. 로봇 팔이 "가지를 목표 지점에 놓기" 과제를 푼다면:

- **Perception**: 카메라 뷰 이미지 + 텍스트 지시를 모델 입력으로 처리
- **Reasoning**: backbone LLM이 텍스트 수준(또는 latent 수준) 추론으로 무엇을 할지 결정
- **Action**: 관절 각도(joint angles) 등을 **special tokens**로 생성 → 실제 세계에서 실행

📌 교수 강조 (강의 발언)

"파이프라인은 결국 LLM과 매우 비슷합니다. 입력 정보를 LLM이 이해할 수 있는 벡터로 변환하고, 전통적인 **next-token prediction**으로 output special tokens를 만드는 것 — tool calling과 ReAct에서 이미 본 패턴이고, 토크나이저가 정의하는 것일 뿐입니다."

Vision Transformer와 패치 임베딩 (patch embedding)

이미지를 LLM에 넣는 가장 직접적인 방법의 토대 (Dosovitskiy et al., ICLR 2021 Oral). raw image $\mathbf{X}_v \in \mathbb{R}^{3 \times H \times W}$ (3 = RGB 채널, H = 높이, W = 너비)를 직접 쓰는 대신:

- 1 이미지를 여러 개의 **패치(patch)**로 분할 (예: $3 \times 3 = 9$ 개) — 패치 수를 L 로 표기
- 2 사전학습된 **vision encoder**(CNN, ViT 등)로 각 패치를 고정 차원 벡터로 변환 → grid features $\mathbf{Z}_v \in \mathbb{R}^{L \times D'}$
- 3 결과: 텍스트의 word embedding 시퀀스와 똑같은 형태의 **벡터 시퀀스**

LLaVA (Large Language and Vision Assistant)

(Liu et al., Visual Instruction Tuning, NeurIPS 2023 Oral; Improved Baselines, CVPR 2024) — **projection-based method**의 대표이자 현재까지 지배적·표준 방식.

$$\mathbf{Z}_v \in \mathbb{R}^{L \times D'} \xrightarrow{\mathbf{W}: \mathbb{R}^{D'} \rightarrow \mathbb{R}^D} \mathbf{H}_v \in \mathbb{R}^{L \times D}$$

vision encoder의 고유 차원 D' 는 LLM 차원 D 와 불일치(mismatch)할 수 있음 → **learnable projector $\mathbf{W}$** (가장 단순하게는 $D' \times D$ 행렬 하나)를 곱해 LLM embedding space로 매핑. $\mathbf{H}_v$ 는 word embedding $\mathbf{H}_q$ 와 같은 subspace에 놓여야 하며, 이것이 학습이 필요한 이유.

2단계 학습 (two sequential steps)

단계	이름	학습 대상	동결(frozen)	목적
Stage 1	Feature alignment	projector $\mathbf{W}$ 만	vision encoder + LLM	무작위 초기화된 projector의 warm-up — 소량의 image-text pairs로 vision feature를 LLM word embedding space에 정렬
Stage 2	Instruction tuning	projector + LLM (jointly)	vision encoder는 계속 frozen	추가 성능 향상을 위한 동시 (simultaneous) 미세조정

데이터: 멀티모달 instruction-following 데이터가 없어서, LLaVA는 (당시 이미지를 못 보던) **GPT-4와 ChatGPT로** conversation / detailed description / complex reasoning 3종의 인공 데이터를 생성해 학습에 사용했다.

교수 강조 (강의 발언)

"왜 vision encoder까지 3개를 다 학습하지 않냐는 질문에는 **golden answer가 없습니다**. 경험적 증거 (empirical evidence)에 달려 있고, 최근 어떤 연구는 반대로 vision encoder+projector만 학습하고 LLM을 동결하기도 합니다."

VideoPoet — projector 패러다임의 일반화

(Kondratyuk et al., **ICML 2024**, Google) — 비디오·오디오·이미지 등 더 많은 모달리티를 이해/생성하는 LLM. 핵심 가정: **각 도메인마다 효율적인 tokenization 방법**이 필요하다. 절차는 LLaVA와 동일: ① 목표 모달리티 결정 → ② 사전학습된 인코더(예: audio encoder)를 빌려옴 → ③ projector + instruction tuning으로 LLM embedding space에 연결.

용어 주의

projector가 변환한 벡터를 흔히 **token**이라 부른다(vision token, audio token, video token).
단, **text token은 정수(integer)**지만 **vision/video token은 벡터** — 고정된 vocabulary와 고정된 word embedding이 없기 때문. 이 구분은 T/F 출제 포인트.

★ 시험 핵심

① 모달리티 결합 공식: *pre-trained encoder* → *patch/grid features* $\mathbf{Z}_v$ → *projector* $\mathbf{W}$ → $\mathbf{H}_v$ (*LLM embedding space*). ② LLaVA 2단계에서 **무엇이 학습되고 무엇이 동결되는지** 단계별로 정확히. ③ vision encoder는 **두 단계 모두 동결**. ④ 학습 데이터는 GPT-4/ChatGPT로 생성.

⚠ 함정 포인트 (T/F 대비)

- "In LLaVA stage 2 (instruction tuning), the vision encoder is fine-tuned together with the LLM." → **False**. The visual encoder is kept *frozen*; only the projector and LLM are fine-tuned jointly.
- "In LLaVA stage 1, the LLM and the projector are trained while the vision encoder is frozen." → **False**. Stage 1 (feature alignment) trains *only the projector*; both vision encoder and LLM are frozen.
- "Vision tokens, like text tokens, are integers from a fixed vocabulary." → **False**. Vision/video tokens are vectors; they have no fixed vocabulary or fixed word embedding.
- "LLaVA collected its multimodal instruction data via human annotation." → **False**. The data was generated using GPT-4 and ChatGPT (text-only at the time).
- "The projector is needed because the vision encoder's output dimension D' may mismatch the LLM's embedding dimension D ." → **True**.

정의

ReAct (Yao et al., ICLR 2023): **reasoning**과 **acting**을 LLM 안에서 통합(integrating reasoning and acting within LLM). action을 **task-specific discrete actions**와 **language**의 **결합(combination)**으로 확장하여, Thought → Action → Observation 사이클을 종료 조건까지 반복. **few-shot prompting**으로 구현(학습 없음).

CoT가 질문과 답 사이에 추론을 끼워 넣듯, ReAct는 **추론 사이에 행동 단계를 끼워(interleave)** 넣는다. CoT는 추론만, action-only는 반성 없는 행동만 하는 데 비해, ReAct는 둘을 동시에 한다. 행동이 환경(웹 등)에서 실행되면 그 결과를 **observation**으로 받아 다음 추론에 반영한다.

- **Wikipedia QA(HotpotQA 등)에서의 행동 3종**: `search`, `lookup`, `finish` — LLM이 호출할 수 있는 Python 함수처럼 생각하면 됨.
- **행동(action)의 일반 정의**: LLM이 기술하더라도 **LLM 바깥에서 외부 호출로 수행되는 모든 행위** — tool calling도, retriever도 하나의 구체적 action이다.

교수 강조 (강의 발언)

"IRCoT와 뭐가 다르냐는 질문 — 높은 수준에서는 같습니다. 하지만 **IRCoT는 action이 retriever로 고정** 된 것이고, ReAct에서 retriever를 가능한 action 중 하나로 넣으면 IRCoT에 대응합니다. **ReAct가 더 일반적인 개념**입니다."

실험 결과와 ReAct ↔ CoT-SC backoff

과제에 따라 효과가 크게 다름 — QA에서는 **CoT-SC(self-consistency: 여러 reasoning paths 생성 후 majority vote)**가 ReAct 단독보다 오히려 좋을 수 있다. 그러나 두 메커니즘을 결합하면 state-of-the-art:

결합 방식	동작	직관
ReAct → CoT-SC	ReAct가 주어진 step 수 안에 답을 반환하지 못하면 CoT-SC로 back off	모델이 과제 처리에 어려움 → 추론에 더 집중해 단일 답 생성

CoT-SC →

n개 경로 중 다수결 득표가 $n/2$ 미만이면

경로들이 한 답에 동의 못 함 = 현재 정보

ReAct

ReAct로 back off

부족 → 환경에서 정보를 더 수집

★ 시험 핵심

① ReAct = reasoning + acting interleave, few-shot prompting 구현. ② backoff 규칙 2개의 조건과 방향(step 초과 → CoT-SC / 득표 $< n/2$ → ReAct)을 정확히 암기. ③ search/lookup/finish 3종 행동. ④ QA에서 CoT-SC가 ReAct 단독보다 좋을 수 있다는 결과.

⚠ 함정 포인트 (T/F 대비)

- "ReAct is implemented by fine-tuning the LLM on reasoning-action trajectories." → **False**. ReAct is implemented by *few-shot prompting*.
- "In the 'CoT-SC → ReAct' strategy, the system backs off to ReAct when ReAct fails to return an answer within the given steps." → **False**. That is 'ReAct → CoT-SC'. CoT-SC → ReAct backs off when the majority vote among n paths occurs less than $n/2$ times.
- "ReAct always outperforms CoT-SC on question answering." → **False**. CoT-SC alone can be much better than ReAct alone on QA; the combination achieves the best results.
- "In ReAct, a retriever call cannot be considered an action." → **False**. Any externally executed operation (retriever, tool call) counts as an action; IRCoT is a special case where the action is fixed to a retriever.

정의

Reflexion (Shinn et al., NeurIPS 2023): **언어적 피드백(linguistic/verbal feedback)**으로 **LLM 에이전트를 강화(reinforce)**하는 프레임워크 — gradient 업데이트 없이 텍스트 수준의 self-reflection을 메모리에 누적하는 "verbal RL". ReAct 저자의 후속 일반화.

구조: Actor / Evaluator / Self-reflection / Memory

- 1 **Actor (LLM)** — policy에 따라 환경(현실 세계, 웹 등)에 행동 → **trajectory**(행동·관찰·추론의 시퀀스, ReAct로 생성) 생성
- 2 **Evaluator (평가 함수)** — trajectory가 충분한지 평가. **heuristic rule** 또는 **LLM 기반 피드백** 모두 가능 (예: hallucination 탐지)
- 3 **Self-reflection (LLM)** — 평가 결과로부터 trajectory에 대한 **고수준(high-level) 텍스트 피드백** 생성 (예: "drawer 1에서 팬을 집으려 했지만 팬은 거기 없었다 → 다음엔 다른 곳을 보라")
- 4 **Memory (long-term, external)** — reflection을 **경험(experience)**으로 저장, 다음 trajectory 생성 시 LLM 입력에 포함

교수 강조 (강의 발언)

"가장 중요한 것은, 모델 파라미터 θ 가 **LLM 파라미터와 메모리를 함께 포괄(encapsulate)**한다는 점입니다. 메모리는 실제로는 텍스트이지만 **개념적으로는 모델 파라미터로 간주**됩니다. 처음에는 메모리가 비어 있고, 반복을 통해 채워지면서 LLM이 과제를 더 잘 수행하게 됩니다."

실험 결과 (반드시 그래프 축 제목 주의!)

- **ALFWorld (sequential decision making)**: 좌측 그래프 Y축 = **해결된 환경 비율**. ReAct도 trial이 늘면 개선되지만, Reflexion(특히 **heuristic 기반 평가 함수**)이 훨씬 높은 success rate. 놀랍게도 **GPT 기반 reflection은 효과적이지만 heuristic보다 나쁨** — 단, 이는 결정적(deterministic)이지 않고 평가 함수 설계·과제에 의존. 핵심: **Reflexion은 평가 함수와 무관하게 ReAct 단독보다 유익**.

- **HotpotQA (reasoning)**: 우측 그래프 Y축 = *hallucination* 오류 사례 비율(실패 사례). ReAct는 약 5 trial 이후 오류율이 **포화(saturate)**해 계산을 더 써도 줄지 않지만, Reflexion은 오류율을 지속적으로 크게 감소시킴. ReAct+Reflexion > ReAct 단독·CoT 단독이며, Reflexion은 CoT에도 유익.

★ 시험 핵심

① "verbal RL" = **weight update가 아니라 텍스트 reflection을 메모리에 누적**. ② 4 구성요소 (Actor/Evaluator/Self-reflection/Memory)와 역할. ③ ALFWorld에서 heuristic > GPT 기반 평가. ④ HotpotQA 그래프는 success가 아니라 **hallucination 실패율** — ReAct는 포화, Reflexion은 계속 감소.

⚠ 함정 포인트 (T/F 대비)

- "Reflexion reinforces the agent by updating the LLM's weights with gradients computed from feedback." → **False**. Reflexion uses *verbal* (linguistic) feedback stored in external memory; no weight update is performed.
- "In Reflexion's ALFWorld experiments, GPT-based self-evaluation outperformed the heuristic-based evaluation function." → **False**. The heuristic evaluation was better, though this is task/design-dependent; both beat ReAct alone.
- "In the HotpotQA analysis, the hallucination-failure rate of ReAct keeps decreasing as trials increase." → **False**. It saturates after about 5 trials; Reflexion keeps reducing it.
- "The evaluator in Reflexion must be an LLM." → **False**. It can be a heuristic rule or an LLM-based feedback function.
- "In Reflexion, the memory starts empty and is filled with self-reflections over iterations." → **True**. Conceptually, θ encapsulates LLM parameters together with this memory.

Planning & Action ③ — Plan-and-Solve & Tree of Thoughts

Plan-and-Solve (PS) Prompting

정의

Plan-and-Solve (Wang et al., ACL 2023): 문제를 풀기 전에 **명시적 계획(explicit planning)**을 먼저 세우게 하는 zero-shot 프롬프팅. zero-shot CoT(Kojima et al., NeurIPS 2022)의 "Let's think step by step" 삽입과 유사하게, "Let's first understand the problem and **devise a plan** to solve the problem. Then, let's **carry out the plan** and solve the problem step by step."을 삽입.

LLM이 먼저 plan을 생성하고 그 plan의 구체적 실행(execution)으로 해답을 만든다. 수학 문제에서 zero-shot CoT보다 전반적으로 더 좋은 성능. 교수 코멘트: 계획의 **필요성 자체는 논쟁의 여지(arguable)**가 있고 최신 LLM에는 암묵적으로 내장되어 있지만, "전체 개요를 잡고 작은 하위 문제로 쪼개는" **분해(decomposition)** 아이디어는 일반적으로 널리 쓰인다 → 그 대표가 ToT.

Tree of Thoughts (ToT)

정의

Tree of Thoughts (Yao et al., NeurIPS 2023 **Oral**): 문제를 **분해(decompose)**하고 LLM으로 **탐색(search)**하는 deliberate problem solving. CoT의 확률성(stochasticity)을 CoT-SC가 다중 경로+투표로 풀었다면, ToT는 추론을 **트리 구조**로 보고 tree search를 도입 — 어려운 부분에 집중하고 쉬운 부분은 건너뛸 수 있음.

ToT의 4가지 구성요소

- 1 **Thought decomposition** — 문제 속성을 활용해 중간 thought step들을 설계·분해 (**via prompting**). Game of 24는 두 수의 연산이므로 3단계, creative writing은 1단계로 충분.
- 2 **Thought generation** — 쿼리+이전 히스토리 기반으로 새 thought(자식 노드) 생성. 방식 2가지: (1) **i.i.d. sampling** (CoT-SC와 유사한 무작위 샘플링), (2) **sequential sampling via**

prompting.

- 3 **State evaluation** — 각 state의 goodness를 LLM prompting으로 추정. 방식 2가지: (1) **Value**: 각 state에 점수(1-10), (2) **Vote**: 같은 layer의 모든 state 중 하나 선택. (예: 남은 숫자 10,13,13으로 24 도달 가능성을 LLM에게 평가시킴 → "불가능" = 낮은 value)
- 4 **Search algorithm** — 다음 state 선택 방법: (1) **BFS(breadth-first search)**, (2) **DFS(depth-first search)**. 효과는 과제 의존적 — 단일 정답 없음.

실험 결과

- **Game of 24** (4개 숫자와 사칙연산으로 24 만들기): ToT는 IO/CoT 대비 압도적 (약 70%대 도달, naive 생성은 한 자릿수~낮은 수준). **b** 는 트리의 너비(breadth) — 단계당 유지하는 자식 노드 수. **b=1인 ToT조차 IO·CoT보다 훨씬 좋음**. 단, **답 하나에 약 100배의 계산** 소모.
- **Creative Writing** (무작위 문장으로 일관성 있는 4문단 생성): ToT > CoT이지만, **단순 IO prompting + iterative refinement만으로도 매우 유사한 성능** → 이 경우 refinement가 훨씬 효율적일 수 있음 (ToT는 다중 탐색으로 계산이 큼).

교수 강조 (강의 발언)

"CoT-SC와 diverse beam search의 차이: DBS는 **diversity penalty**를 명시적으로 추가하지만, self-consistency는 **전적으로 샘플링의 무작위성에 의존**합니다. 이론상 최악엔 다 같은 출력일 수 있지만 실제로는 충분히 다양합니다." / "이 분야에 관심 있으면 ReAct 1저자 **Shunyu Yao**(Reflexion·ToT의 교신저자, 전 OpenAI)의 논문들을 강력 추천합니다."



비교표: ReAct vs Reflexion vs Plan-and-Solve vs ToT

구분	ReAct (ICLR 2023)	Reflexion (NeurIPS 2023)	Plan-and-Solve (ACL 2023)	ToT (NeurIPS 2023 Oral)
핵심 아이디어	reasoning + acting interleave	verbal RL: trajectory 평가 → 텍스트 reflection → memory	풀기 전 명시적 plan 생성	thought 트리 분해 + 탐색

구현 방식	few-shot prompting	Actor/Evaluator/Self- reflection + external memory	zero-shot prompt 1문 장 교체	prompting + BFS/DFS 탐색
환경 상호 작용	O (action→observation)	O (ReAct trajectory 사용)	X (순수 프 로프팅)	X (내부 탐색)
피드 백/ 평가	없음 (observation만)	evaluator(heuristic 또는 LLM) + self-reflection	없음	state evaluation (value 1-10 / vote)
반복 학습 효과	trial 증가로 개선되나 포화	iteration마다 memory 누적 으로 자기 개선	단발성	탐색 내 backtrack 가능
대표 과제	HotpotQA, 웹 상호작용	ALFWorld, HotpotQA, 프로 그래밍	수학 추론	Game of 24 (~70%), Creative Writing
비용	낮음~중간	중간 (반복 횟수에 비례)	낮음	매우 높음 (~100x)

★ 시험 핵심

ToT의 4 구성요소 이름과 각각의 **2가지 옵션**(generation: i.i.d./sequential, evaluation: value/vote, search: BFS/DFS)을 매칭할 수 있어야 한다. Game of 24가 3단계로 분해되는 이유 (수치 연산은 항상 두 수를 취함). Creative writing에서는 **IO+refinement**로도 **충분**하다는 한계.

⚠ 함정 포인트 (T/F 대비)

- "Plan-and-Solve requires few-shot demonstrations of plans." → **False**. It is a zero-shot prompting method, analogous to inserting "let's think step-by-step" in zero-shot CoT.

- "In ToT, state evaluation is performed by a separately trained value network." → **False**. It is done via LLM prompting — value scoring (1-10) or voting among states in the same layer.
- "ToT outperformed all baselines on creative writing by a large margin." → **False**. Simple input-output prompting with iterative refinement achieved very similar performance, and is far more efficient.
- "ToT with $b=1$ is equivalent to chain-of-thought." → **False**. Even $b=1$ ToT (one child per step, with evaluation/search) is much better than IO or CoT on Game of 24.
- "ToT uses Monte-Carlo Tree Search." → **False**. The paper considers BFS and DFS.

Multi-modal Action — Diffusion (MetaQueries, BLIP3-o) 결합

지금까지의 action은 텍스트(함수 호출 등) 기반이었다. 그러나 이미지·비디오·오디오 생성처럼 **연속 도메인 (continuous domains)의 출력**이 필요한 과제에서는 텍스트 기반 행동만으로는 제한적이다.

📖 핵심 명제

Autoregressive modeling is not the best choice for generation in other domains — diffusion model(Sohl-Dickstein et al., ICML 2015; Ho et al., DDPM, NeurIPS 2020)이 vision & audio 같은 **연속 도메인에서 일반적으로 더 낫다**고 알려져 있다. diffusion = 무작위 노이즈로부터 목표 데이터를 재구성하도록 학습하는 생성 모델.

따라서 **diffusion을 LLM에 통합(unifying diffusion into LLM)**하는 연구가 활발하다:

- **Diffusion-VLA** (Wen et al., ICML 2025) — 로봇 파운데이션 모델에 diffusion 결합 (self-generated reasoning)
- **MetaQueries** (Pan et al., arXiv 25.04, **Meta AI**) — vision-language model 위에 **diffusion block**을 올려 이미지를 생성 (modality 간 transfer)
- **BLIP3-o** (Chen et al., arXiv 25.05, **Salesforce**) — fully open unified multimodal model, 이미지 생성을 위해 **diffusion layer** 포함

⚠️ 함정 포인트 (T/F 대비)

- "Autoregressive next-token prediction is generally the best choice for generation in every modality." → **False**. Diffusion models are generally known to be better at continuous domains (vision & audio).
- "MetaQueries and BLIP3-o generate images purely autoregressively without any diffusion component." → **False**. Both include a diffusion block/layer on top of the (vision-)language model.

7 Memory ① — MemGPT (LLMs as Operating Systems)

왜 Memory가 필요한가

Agentic workflow(ToT의 다중 탐색, Reflexion의 반복 trajectory 등)는 **입력 컨텍스트 누적을 급격히 증가**시킨다. 모든 컨텍스트를 입력에 넣는 것은 두 가지 이유로 **infeasible**: ① **context window limit**, ② transformer의 이차적(quadratic) 성질로 인한 **inference cost** 증가. → 해법: **external memory & selective retrieval**. (Reflexion의 메모리는 모든 reflection을 항상 append하는 한계 → 더 나은 관리 필요)

정의

MemGPT (Packer et al., arXiv 23.10): **현대 컴퓨터 OS에서 영감을 받은(OS-inspired)** 메모리 설계. **in-context memory = main memory/physical memory/RAM**, **external memory = disk memory/disk storage**에 대응시키고, OS 메모리 시스템의 동작을 모방하는 연산들을 도입. top-tier 학회 미출판이지만 이 분야에서 가장 영향력 있는 연구 중 하나.

Main context의 3분할 (contiguous sections)

구역	성질	내용
System instructions	read-only (static)	제어 흐름(control flow)·메모리 사용법(함수들을 API처럼 기술). LLM의 system prompt에 해당
Working context	읽기/쓰기	key facts, preferences, 기타 사용자 정보 저장 — 여러 대화 세션에 걸친 요약
FIFO queue	롤링	agent↔user 메시지 + system messages(예: memory warnings) + function call 의 입력·출력

⚠ 용어 구분

- FIFO queue에 들어가는 "system messages"(memory warning 등, 환경에서 옴)는 "system instructions"(read-only 시스템 프롬프트)와 **다른 것**이다 — 혼동 유도 출제 가능.

Queue Manager (QM)와 eviction policy

- 1 **새 메시지 처리** — QM이 FIFO queue에 append하고 LLM 응답을 trigger. **들어온 메시지와 LLM 출력 모두 recall storage(message database)에 기록.**
- 2 **70% 초과 (warning threshold)** — prompt tokens가 임계값(70%)을 넘으면 QM이 **경고용 system message를 큐에 삽입** → LLM이 (system instructions의 함수 설명을 보고) 정보를 **working context나 archival storage(외부 메모리)에 저장하는 함수를 호출**할 수 있음.
- 3 **100% 초과 (context window limit)** — 메시지의 특정 부분(50%)을 **evict**. 단, 그냥 버리지 않고 기존 요약+축출 메시지로 **recursive summary를 생성**해 FIFO queue 앞에 prepend.

Function executor

컨텍스트와 메모리 간 **데이터 이동(data movement)을 orchestrate**. 모든 함수는 system instructions에 기술되어 있고, **LLM이 in-context learning으로 적응적으로(adaptively) 호출을 결정**한다. 예: 사용자 생일(2월 7일)·남자친구 이름을 working context에 추가, out-of-context 데이터를 archival storage에서 검색.

실험 결과

- **Deep memory retrieval**: 여러 세션·턴 이후 과거 대화의 사실을 회상해야 하는 질문 — 베이스라인 (최근 대화만 trimming해 표시)은 한계 밖 정보를 회상 못 하지만 MemGPT는 검색으로 성공.
- **Open-ended QA**: 베이스라인은 검색 문서 수가 context limit를 넘으면 더 많은 문서가 도움이 안 되지만, **MemGPT는 context window limit에 묶이지 않아 꾸준한(steady) 성능.**

★ 시험 핵심

숫자 3개: **70%(warning) / 100%(eviction trigger) / 50%(evict 비율)** + recursive summary. OS 비유 매핑(in-context↔RAM, external↔disk). 메모리 관리가 **LLM의 function call**로 이루어진다는 점(QM은 레이어, 결정은 LLM의 in-context learning).

⚠ 함정 포인트 (T/F 대비)

- "MemGPT evicts 50% of messages as soon as prompt tokens exceed 70% of the context window." → **False**. At 70% it only inserts a *warning* system message; eviction (50% of messages) happens when exceeding 100% (the context window limit).
- "Evicted messages in MemGPT are simply discarded." → **False**. A recursive summary is generated from the existing summaries and the evicted messages.
- "In MemGPT, working context corresponds to external (disk) memory." → **False**. Working context is part of the *main context* (in-context memory ↔ RAM); archival/recall storage corresponds to disk.
- "Memory management in MemGPT is performed by a separately fine-tuned controller network." → **False**. The LLM itself adaptively decides via function calls, guided by instructions in the system instructions (in-context learning).
- "System instructions in MemGPT are read-only and static." → **True**.

8 Memory ② — A-Mem (Agentic Memory)

정의

A-Mem (Xu et al., A-MEM: Agentic Memory for LLM Agents, NeurIPS 2025): MemGPT의 메모리 구조는 **정적(static)**(OS 모방) — 대신 **LLM이 메모리를 동적으로(dynamically) 조직**(구축·연결·수정·진화)하는 **agentic memory**를 제안. 3단계: **(1) Note construction, (2) Link generation, (3) Memory evolution.**

(1) Note construction — 메모리 노트의 7요소

Memory note

$$m_i = \{c_i, t_i, K_i, G_i, X_i, e_i, L_i\}$$

c_i : **original content** (원시 내용 — 사용자 쿼리 또는 LLM 생성) · t_i : **timestamp** · K_i, G_i, X_i : **LLM-generated keyword, tags, description** (프롬프트 P 에 $c_i \oplus t_i$ (결합, concatenation)를 넣어 생성 — 내용에 대한 고수준 메타데이터) · e_i : **text encoder의 vector representation** (content와 K_i, G_i, X_i 를 결합해 인코딩) · L_i : **set of linked memories** (의미가 유사한 메모리들과의 연결).

(2) Link generation — 새 노트 m_n 추가 시

- 1 먼저 임베딩의 **cosine similarity**로 **top-k most relevant memories** 식별 (dense retriever와 동일한 아이디어) → m_n 의 neighbor set
- 2 그 k개 후보를 m_n 과 함께 **LLM에 prompting하여 link set 생성** — 어떤 메모리가 실제로 관련되는지는 LLM이 결정

교수 강조 (강의 발언)

"모든 기존 메모리를 LLM에 보여주는 것이 더 넓은 시야겠지만 **infeasible**하므로, top-k 검색은 그 **중간해법(intermediate solution)**입니다. 또한 K_i, G_i, X_i 를 임베딩에 함께 쓰는 설계는 raw text보다 낫다고 저자들이 주장하지만, **논쟁의 여지가 있는(arguable) 설계**입니다."

(3) Memory evolution

새 메모리가 들어오면, neighbor set 안의 각 메모리 m_j 의 **context(description), keywords, tags**가 **선택적으로(optionally) 업데이트**된다 (예: "사용자가 노벨 물리학상·화학상·경제학상을 물었고 이제 또 물리학을 물음 → 이 분야에 더 관심" 식의 고수준 설명 갱신). **진화된 메모리는 업데이트된 경우 원본을 대체(replace)**한다.

Memory retrieval & 평가

쿼리 q 를 동일한 text encoder로 벡터화 → cosine similarity로 **k 개의 most relevant memories**를 검색 → 일반적인 RAG처럼 입력 프롬프트에 append.

평가: LoCoMo QA (Maharana et al., **ACL 2024**; Long-term Conversational Memory) — single-hop / multi-hop / temporal reasoning / commonsense / 기타 5범주. 기존 벤치마크 대비 대화당 평균 turn 수·세션 수·timespan이 훨씬 김. 결과: **A-Mem이 기존 메모리 기반 접근법(MemGPT 포함)보다 일관되게(consistently) 우수.**

비교표: MemGPT vs A-Mem

구분	MemGPT (arXiv 23.10)	A-Mem (NeurIPS 2025)
설계 영감	OS 메모리 계층 (RAM ↔ disk)	LLM 특화 agentic memory
메모리 구조	정적(static) — 구역(system instructions/working context/FIFO queue) 고정	동적(dynamic) — LLM이 노트를 만들고 연결·진화시킴
단위	메시지/요약 (FIFO queue + recall/archival storage)	memory note m_i (7요소: c, t, K, G, X, e, L)
관리 주체	Queue Manager(레이어) + LLM function call	LLM 자체 (link generation, memory evolution)
기존 메모리 갱신	working context 수정, eviction + recursive summary	이웃 노트의 description/keywords/tags가 진화하며 원본 대체
검색	function call로 recall/archival storage 검색	쿼리 임베딩 cosine similarity top-k → RAG식 append

성능
(LoCoMo)

베이스라인

MemGPT보다 일관되게 우수

★ 시험 핵심

노트 7요소 기호와 의미 매칭(c_i =원본 내용, t_i =timestamp, $K/G/X$ =LLM 생성 keyword/tags/description, e_i =인코더 벡터, L_i =링크 집합). link generation = **cosine top-k 후보** → **LLM 결정**의 2단계. evolution은 **optional** 업데이트. LoCoMo는 ACL 2024.

⚠ 함정 포인트 (T/F 대비)

- "In A-Mem, links between memories are determined purely by cosine similarity." → **False**. Cosine similarity only retrieves top-k candidates; the LLM is then prompted to generate the actual link set.
- " K_i, G_i, X_i in A-Mem are written by human annotators." → **False**. They are LLM-generated keyword, tags, and description.
- "Memory evolution in A-Mem always rewrites every memory in the system whenever a new note arrives." → **False**. Only memories in the neighbor set are (optionally) updated; the evolved memory replaces the original only if updated.
- "The embedding e_i is computed from the original content alone." → **False**. The content is concatenated with the LLM-generated metadata (K, G, X) before encoding.
- "A-Mem organizes memory using a fixed OS-like hierarchy as in MemGPT." → **False**. A-Mem's point is to replace the static structure with dynamic, LLM-driven (agentic) organization.

에이전트의 단위: 같은 LLM도 다른 에이전트가 될 수 있다

- **Persona-based prompting**: 같은 LLM에 다른 시스템 프롬프트(예: "탐색 담당" vs "비평 담당")
→ **역할(role)이 다르면 다른 에이전트**. 단일 LLM으로 여러 에이전트를 만드는 가장 표준적·편리한 방법
(예: MDAgents).
- **Specialized LLM**: 특화 도메인이면 다른 모델 사용 (예: 의료의 **MedGemma**, Google). 단, 필수는
아님 — agentic plan 설계가 좋다면 단일 LLM+역할 분담으로 충분.

MDAgents — 의료 의사결정 멀티에이전트

정의

MDAgents (Kim et al., An Adaptive Collaboration of LLMs for Medical Decision-Making, **NeurIPS 2024 Oral**): 의료 의사결정에서 **과제 난이도(complexity check)**에 따라 에이전트 조직을 적응적으로(**adaptive**) 구성하는 시스템.

- 1 **Complexity check** — orchestration layer(LLM checker)가 과제 난이도 판정
- 2 **Low(쉬움)** → 단일 에이전트(**solo**)가 일반 QA로 해결
- 3 **Moderate(중간)** → 서로 다른 역할의 에이전트들로 **팀(group) 구성**: 전문가(experts)들이 moderator와 함께 initial assessment → collaborative discussion → 최종 결정
- 4 **High(어려움)** → 여러 팀을 만들어 팀 간 debate 또는 과제 분담 (계층 구조)

각 전문가는 GPT/Gemini 같은 **단일 LLM이지만 프롬프트(역할)가 다름**. 결과: MDAgents > 일반 단일 에이전트 및 멀티에이전트 베이스라인. 논문은 멀티에이전트를 **single model 기반 vs multiple models 기반**으로 구분해 비교.

핵심 구성요소 (1): Structure (interaction type)

Anthropic의 multi-agent research system 사례에서: 쉬우면 single agent, 중간이면 single team, 어려우면 multiple teams + hierarchical structure. 그 외 형태: **single / voting / debate / meta-agent / reconfiguration** 등.

📖 교수 강조 (강의 발언)

"각 논문의 특정 구조를 외우는 게 중요한 게 아니라, **멀티에이전트 시스템에서 역할(role)이 어떻게 정의되고 상호작용(interaction)이 어떻게 정의되는지**가 더 중요합니다."

Orchestration 기반 구조 (Claude 사례)

가장 강력한 LLM을 중앙의 **lead agent(orchestrator)**로 두고, 과제에 따라 **sub-agent에게 할당**하는 구조 (Anthropic 등에서 널리 도입). 워크플로: 사용자 쿼리 → lead orchestrator(lead researcher) → sub-agents 호출·해결 → orchestration layer가 결과를 모아 단일 답 생성. 자연스러운 구성:

orchestrator = Opus, sub-agents = Haiku — 비용 절약이 큰 동기이며, Haiku가 못 풀면 Sonnet/Opus 같은 더 능력 있는 모델을 호출하는 식의 비용 제어가 가능.

핵심 구성요소 (2): Goal

- **성능 개선** — 가장 직접적인 목표
- **비용 절약 = routing** — BEST-Route (Ding et al., ICML 2025), IRT-Router (Song et al., ACL 2025)
- **그 외** — tabular data 피처 생성 (Nam et al., NeurIPS 2024), personalization (Kim et al., NAACL 2025), **jailbreak** (Koo et al., Align to Misalign, ICLR 2026 — 교수 연구실 연구: 멀티에이전트로 jailbreak 프롬프트 자동 생성)

★ 시험 핵심

- ① MDAgents의 **complexity check → solo/team/teams** 적응 구조 (NeurIPS 2024 Oral).
- ② 같은 LLM + 다른 프롬프트(persona) = 다른 에이전트로 인정. ③ orchestration: 강한 모델이 lead, 싹 모델이 sub (Opus/Haiku), 동기는 성능+비용. ④ 멀티에이전트의 2대 구성요소: **Structure(interaction type)와 Goal**.

⚠ 함정 포인트 (T/F 대비)

- "Two agents must be different LLMs to count as a multi-agent system." → **False**.
The same LLM with different prompts/personas (different roles) counts as different

agents (e.g., MDAgents).

- "MDAgents always forms multiple teams of experts regardless of task difficulty." → **False**. It adaptively checks complexity: solo agent for easy, one group for moderate, multiple teams for hard tasks.
- "In orchestration-based systems, the cheapest model is used as the lead agent to save cost." → **False**. The most capable model (e.g., Opus) is the lead/orchestrator; cheaper models (e.g., Haiku) serve as sub-agents.
- "Routing in multi-agent systems refers only to network packet routing." → **False**. Routing = adaptively choosing which LLM handles a query, mainly to save cost (BEST-Route, IRT-Router).
- "The goal of a multi-agent system is limited to improving task performance." → **False**. Goals include cost saving (routing), tabular data, personalization, and even jailbreak.

Q 예상문제 (직접 기출 없음 — 중간고사 이후 범위)

※ #11은 2025 중간 기출에 직접 출제된 바 없음(시험 범위 밖). 아래는 기출 문제·형식·난이도를 따른 예상문제. T/F 는 정답 3점·오답 0점·무응답 1점 — 확신 없으면 비워두는 것도 전략.

True / False

Q1. (True/False) 예상문제

ReAct extends the action space of an LLM agent to a combination of task-specific discrete actions and language, and is implemented by few-shot prompting rather than fine-tuning.

▼ 정답 & 해설 보기

True. ReAct integrates reasoning and acting within the LLM by extending action as a combination of task-specific discrete actions and language, and the slides explicitly state "it is implemented by few-shot prompting."

Q2. (True/False) 예상문제

In the combined strategy "ReAct → CoT-SC", the system starts with CoT-SC and backs off to ReAct when the majority vote among n reasoning paths occurs less than $n/2$ times.

▼ 정답 & 해설 보기

False. The description is reversed. "ReAct → CoT-SC" starts with ReAct and backs off to CoT-SC when ReAct fails to return an answer within the given steps. The voting condition ($< n/2$) belongs to "CoT-SC → ReAct".

Q3. (True/False) 예상문제

Reflexion improves the agent over trials by storing verbal self-reflections in an external memory, without performing any gradient-based update of the LLM's weights.

▼ 정답 & 해설 보기

True. Reflexion is "verbal reinforcement learning": trajectories are evaluated, high-level linguistic feedback (reflection) is generated and accumulated in long-term memory, and this text — not a weight update — improves subsequent trajectories. Conceptually, θ encapsulates the LLM parameters together with the memory.

Q4. (True/False) 예상문제

In the Reflexion experiments on ALFWorld, the GPT-based self-evaluation function achieved a higher success rate than the heuristic-based evaluation function.

▼ 정답 & 해설 보기

False. Somewhat surprisingly, GPT-based reflection was effective but *worse* than the heuristic-based evaluation. (Note: this is not deterministic — it depends on evaluation-function design and the task — but both variants beat ReAct alone.)

Q5. (True/False) 예상문제

In Tree of Thoughts, state evaluation can be performed either by scoring each state on a 1-10 scale (value) or by selecting one state among all states in the same layer (vote), both via LLM prompting.

▼ 정답 & 해설 보기

True. ToT estimates the goodness of each state via LLM prompting with two options: (1) Value — scoring each state (1-10); (2) Vote — selecting one among all states in the same layer. No trained value network is used.

Q6. (True/False) 예상문제

In LLaVA's two-stage training, the pre-trained vision encoder is kept frozen in both stages; stage 1 trains only the projector, and stage 2 fine-tunes the projector and the LLM jointly.

▼ 정답 & 해설 보기

True. Stage 1 (feature alignment) trains the randomly initialized projector W on small image-text pairs with everything else frozen; stage 2 (instruction tuning) keeps the visual encoder frozen and fine-tunes the projector and LLM simultaneously.

Q7. (True/False) 예상문제

In MemGPT, when prompt tokens exceed 70% of the context window, the queue manager immediately evicts 50% of the messages in the FIFO queue.

▼ 정답 & 해설 보기

False. At the 70% threshold, the queue manager only inserts a warning system message (so the LLM may call functions to store information into working context or archival storage). Eviction of 50% of messages happens only when the context window limit (100%) is exceeded, followed by generating a recursive summary of the evicted messages.

Q8. (True/False) 예상문제

In A-Mem, the link set of a new memory note is generated purely by cosine similarity between embeddings, without involving the LLM.

▼ 정답 & 해설 보기

False. Cosine similarity is only used to identify the top-k most relevant candidate memories (an intermediate solution, since showing all memories to the LLM is infeasible). The LLM is then prompted to decide which of those candidates are actually linked, generating the link set.

Q9. (True/False) 예상문제

Two agents in a multi-agent system can be instances of the same LLM, as long as they are given different prompts (e.g., personas) that assign them different roles.

▼ 정답 & 해설 보기

True. Persona-based prompting is the most standard way to define multiple agents from a single LLM (e.g., MDAgents' experts are single LLMs like GPT/Gemini with different prompts). Specialized LLMs (e.g., MedGemma) are an option, not a requirement.

Q10. (True/False) 예상문제

Diffusion models are generally known to be better than autoregressive modeling for generation in continuous domains such as vision and audio, which motivates unifying diffusion into LLMs (e.g., MetaQueries, BLIP3-o).

▼ 정답 & 해설 보기

True. The slides state that autoregressive modeling is not the best choice for generation in other domains, and diffusion is generally better at continuous domains (vision & audio); MetaQueries (Meta AI) and BLIP3-o (Salesforce) add diffusion blocks/layers on top of (vision-)language models.

Pick all that are correct (any wrong answer will lead to 0 points)

Q11. (Pick all that are correct) 예상문제

Which of the following are true about the key concepts of LLM agents?

- a. An LLM agent is characterized as LLM + planning & action + tools + memory.
- b. These agentic capabilities are simply improved by scaling up model and data size.
- c. An intelligent agent perceives its environment, takes actions, and can improve its own performance by repeating this procedure.
- d. Memory is arguably the hardest capability to integrate inside the LLM itself, so it is often handled by external systems.

▼ 정답 & 해설 보기

정답: (a), (c), (d)

- (a) Correct. This is the definition used throughout the chapter (Lilian Weng's framework).
- (b) Incorrect. The slides explicitly say these capabilities are *not* simply improved by just scaling-up model & data size; new training methods, data types, or harnesses are needed.
- (c) Correct. Perception, (implicit) reasoning, action + self-improvement is the definition of an intelligent agent.
- (d) Correct. The professor noted memory is probably the hardest part to integrate into the LLM (handled externally), whereas tool use can naturally be integrated via training.

Q12. (Pick all that are correct) 예상문제

Which of the following are true about LLaVA and projection-based multimodality?

- a. An image $X_v \in \mathbb{R}^{3 \times H \times W}$ is first mapped by a pre-trained vision encoder into grid features $Z_v \in \mathbb{R}^{L \times D'}$, where L is the number of patches.
- b. The learnable projector W maps $\mathbb{R}^{D'}$ to $\mathbb{R}^D$ so that the visual features lie in the LLM's word embedding space.
- c. LLaVA's multimodal instruction-following dataset was collected by GPT-4 and ChatGPT for conversation, detailed description, and complex reasoning.
- d. Vision tokens are integers drawn from a fixed vocabulary, exactly like text tokens.
- e. VideoPoet applies the same projection idea to more modalities, assuming an efficient tokenization method for each domain.

▼ 정답 & 해설 보기

정답: (a), (b), (c), (e)

- (a) Correct. This is the ViT-style patch embedding step.
- (b) Correct. The projector resolves the dimension mismatch (D' vs D) so H_v lies in the same subspace as the word embeddings.

- (c) Correct. GPT-4/ChatGPT (text-only at the time) generated the artificial multimodal data.
- (d) Incorrect. Vision/video tokens are vectors, not integers; they have no fixed vocabulary or fixed word embedding, unlike text tokens.
- (e) Correct. VideoPoet (ICML 2024) borrows pre-trained encoders per modality and connects them via projectors — the standard practice.

Q13. (Pick all that are correct) 예상문제

Which of the following are true about ReAct and Reflexion?

- a. In ReAct's Wikipedia-based QA setting, the possible actions are search, lookup, and finish.
- b. On QA tasks, ReAct alone always outperforms CoT with self-consistency.
- c. In Reflexion, a trajectory generated by the actor is evaluated (by a heuristic rule or an LLM), and a self-reflection model produces high-level verbal feedback stored in long-term memory.
- d. On HotpotQA, the proportion of hallucination failure cases for ReAct saturates after about 5 trials, while Reflexion keeps reducing it.
- e. IRCot can be seen as a special case of ReAct where the action is fixed to a retriever.

▼ 정답 & 해설 보기

정답: (a), (c), (d), (e)

- (a) Correct. The three actions are defined like Python functions the LLM can call.
- (b) Incorrect. CoT-SC alone can be much better than ReAct alone on QA; the combined backoff strategies achieve the best results.
- (c) Correct. This is the actor / evaluator / self-reflection / memory structure.
- (d) Correct. Note the y-axis of that figure is the failure (hallucination) rate, not the success rate.

- (e) Correct. The professor explicitly made this connection: ReAct is the more general conceptual idea.

Q14. (Pick all that are correct) 예상문제

Which of the following are true about Tree of Thoughts (ToT)?

- a. Thought decomposition designs intermediate thought steps via prompting, leveraging properties of the problem (e.g., 3 steps for Game of 24).
- b. Thought generation can use (1) i.i.d. sampling similar to CoT-SC or (2) sequential sampling via prompting.
- c. The search over the tree is performed with Monte-Carlo Tree Search guided by a learned reward model.
- d. On Game of 24, even ToT with $b=1$ outperforms input-output and chain-of-thought prompting, but ToT consumes roughly 100× more computation per answer.
- e. On creative writing, simple input-output prompting with iterative refinement achieved performance very similar to ToT.

▼ 정답 & 해설 보기

정답: (a), (b), (d), (e)

- (a) Correct. Numerical operations always take two numbers, so three reasoning steps are needed for Game of 24; creative writing needs only one step.
- (b) Correct. These are the two thought-generation options.
- (c) Incorrect. ToT uses BFS or DFS (whose effectiveness is task-dependent), and state evaluation is done by LLM prompting (value 1-10 or vote), not a learned reward model or MCTS.
- (d) Correct. b is the breadth (number of child nodes per step); even $b=1$ beats IO/CoT, at ~100× compute cost.
- (e) Correct. This shows IO + refinement can be far more efficient than ToT for that task.

Q15. (Pick all that are correct) 예상문제

Which of the following are true about MemGPT?

- a. In-context memory corresponds to main memory (RAM), while external memory corresponds to disk storage, following an OS-inspired design.
- b. The main context consists of system instructions (read-only), working context, and a FIFO queue.
- c. Both incoming messages and LLM outputs are written into recall storage by the queue manager.
- d. Data movement between context and memory is decided by the LLM itself via function calls, guided by descriptions in the system instructions (in-context learning).
- e. On open-ended QA, MemGPT's performance degrades sharply once the number of retrieved passages exceeds the context window limit.

▼ 정답 & 해설 보기

정답: (a), (b), (c), (d)

- (a) Correct. This is the core OS analogy of MemGPT.
- (b) Correct. The FIFO queue holds agent-user messages, system messages (e.g., memory warnings), and function call inputs/outputs.
- (c) Correct. The queue manager appends new messages to the FIFO queue, triggers the LLM, and writes both directions into the recall storage (message database).
- (d) Correct. The function executor orchestrates data movement; the LLM adaptively decides which functions to call.
- (e) Incorrect. That describes the *baseline*. MemGPT is not bounded by the context window limit and achieves steady performance.

Q16. (Pick all that are correct) 예상문제

Which of the following are true about A-Mem?

- a. A memory note m_i consists of the original content c_i , timestamp t_i , LLM-generated keyword/tags/description (K_i, G_i, X_i) , a vector representation e_i , and a set of linked memories L_i .

- b. When a new note arrives, the top-k most relevant memories are first identified by embedding similarity, and then an LLM is prompted to generate the link set.
- c. Memory evolution may update the context, keywords, and tags of memories in the neighbor set, and the evolved memory replaces the original if updated.
- d. A-Mem was evaluated on the LoCoMo QA dataset (ACL 2024) and performed consistently better than previous memory-based approaches, including MemGPT.
- e. Like MemGPT, A-Mem organizes memory with a static, OS-inspired structure.

▼ 정답 & 해설 보기

정답: (a), (b), (c), (d)

- (a) Correct. These are the seven elements of a memory note.
- (b) Correct. Top-k retrieval is the feasible intermediate solution; the LLM makes the final linking decision.
- (c) Correct. The update is optional and applies to the neighbor set.
- (d) Correct. LoCoMo (long-term conversational memory; single-hop, multi-hop, temporal reasoning, commonsense, etc.) has far more turns/sessions and a longer timespan than prior benchmarks.
- (e) Incorrect. A-Mem's key idea is to organize memories *dynamically* in an agentic way, in contrast to MemGPT's static structure.

Q17. (Pick all that are correct) 예상문제

Which of the following are true about multi-agent systems?

- a. In MDAgents, an LLM first checks the complexity of the medical task and adaptively chooses between a solo agent (easy), a single group of experts with a moderator (moderate), and multiple teams (hard).
- b. In an orchestration-based system, the most capable model serves as the lead agent/orchestrator and assigns tasks to cheaper sub-agents (e.g., Opus orchestrating Haiku).
- c. Saving inference cost by adaptively selecting which model answers a query is often called routing.

- d. A multi-agent system requires each agent to be a differently pre-trained LLM.
- e. Multi-agent systems have also been used for goals beyond accuracy, such as tabular feature generation, personalization, and jailbreak.

▼ 정답 & 해설 보기

정답: (a), (b), (c), (e)

- (a) Correct. This is MDAgents' adaptive collaboration hierarchy (NeurIPS 2024 Oral): complexity check → initial assessment → collaborative discussion → final decision for the group case.
- (b) Correct. Cost saving and capability-based escalation (calling Sonnet/Opus when Haiku fails) are key motivations for orchestration.
- (c) Correct. E.g., BEST-Route (ICML 2025), IRT-Router (ACL 2025).
- (d) Incorrect. Agents can be the same LLM with different prompts/personas; specialized models (e.g., MedGemma) are optional.
- (e) Correct. Examples from the slides: optimized feature generation for tabular data, few-shot personalization, and automatic LLM jailbreak with meta-optimized judges.

3분 요약

개념	한 줄 정리
LLM Agent	LLM(brain) + Planning & Action + Tools + Memory — scaling만으로는 안 됨
LLaVA	vision encoder → grid features Z_v → projector W → H_v (LLM 공간); 2단계 학습 (① projector만 ② projector+LLM, encoder는 항상 frozen), 데이터는 GPT-4 생성
VideoPoet / VLA	모달리티마다 encoder+projector = 표준 패턴; VLA는 action을 special token으로 출력 (OpenVLA)
ReAct	reasoning+acting interleave, few-shot prompting; step 초과→CoT-SC, 득표 < n/2 → ReAct backoff
Reflexion	verbal RL: Actor→Evaluator(heuristic>GPT)→Self-reflection→Memory 누적, weight update 없음
Plan-and-Solve	zero-shot로 "plan 먼저, 그다음 실행" 프롬프트 삽입
ToT	분해→생성(i.i.d./sequential)→평가(value/vote)→탐색(BFS/DFS); Game of 24 압승, 비용 ~100x
Multi-modal Action	연속 도메인은 diffusion이 우세 → LLM+diffusion 통합 (MetaQueries, BLIP3-o)
MemGPT	OS 비유(RAM↔in-context, disk↔external); main context = system instructions/working context/FIFO queue; 70% 경고→100%에 50% evict+recursive summary; 관리는 LLM function call
A-Mem	agentic(동적) 메모리: note {c,t,K,G,X,e,L} → cosine top-k+LLM link → evolution; LoCoMo에서 MemGPT 능가
Multi-agent	persona만 달라도 다른 에이전트; MDAgents = complexity check→solo/group/teams; orchestration = 강한 lead(Opus)+싼 sub(Haiku);

Goal = 성능·routing(비용)·기타

[← 전체 목차](#)

[다음 장 →](#)